

Security Audit

Silverswap 4th (DCA)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	9
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	24
Conclusion	25
Our Methodology	26
Disclaimers	28
About Hashlock	29

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Silverswap team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

SilverSwap.io is a decentralized exchange (DEX) that enables users to trade and swap cryptocurrencies in a secure, non-custodial manner. The platform is built to facilitate decentralized trading without the involvement of intermediaries, offering users full control over their digital assets. By operating on blockchain technology, SilverSwap ensures transparency and security in transactions, while promoting a peer-to-peer trading model. Users can participate in liquidity pools, where they provide liquidity in exchange for potential rewards, helping to maintain the platform's fluidity and enabling seamless swaps of various digital assets.

Project Name: Silverswap

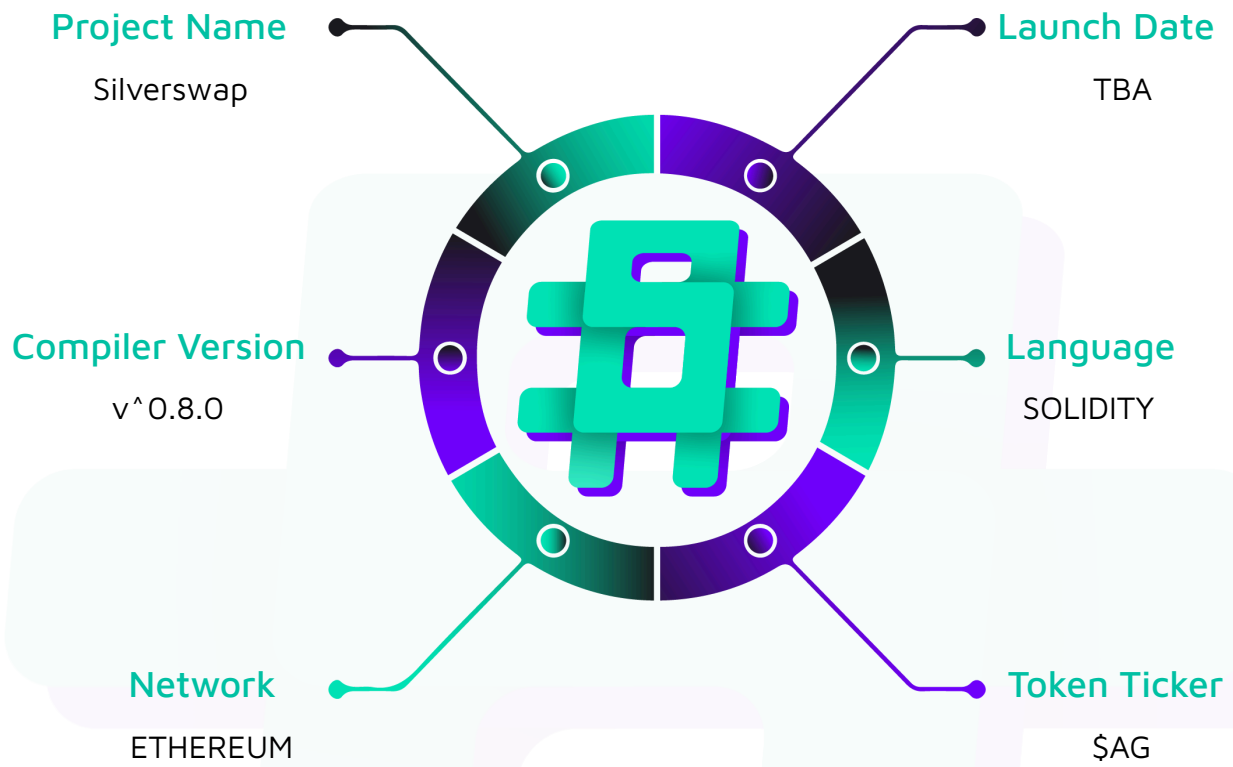
Project Type: Defi

Compiler Version: ^0.8.0

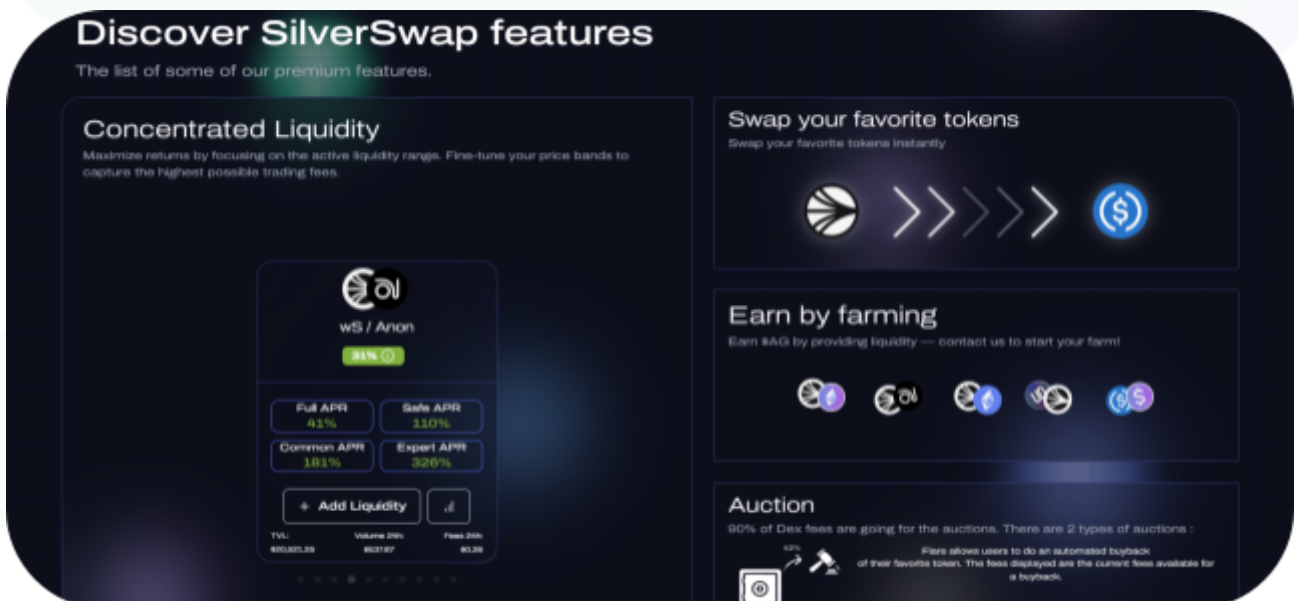
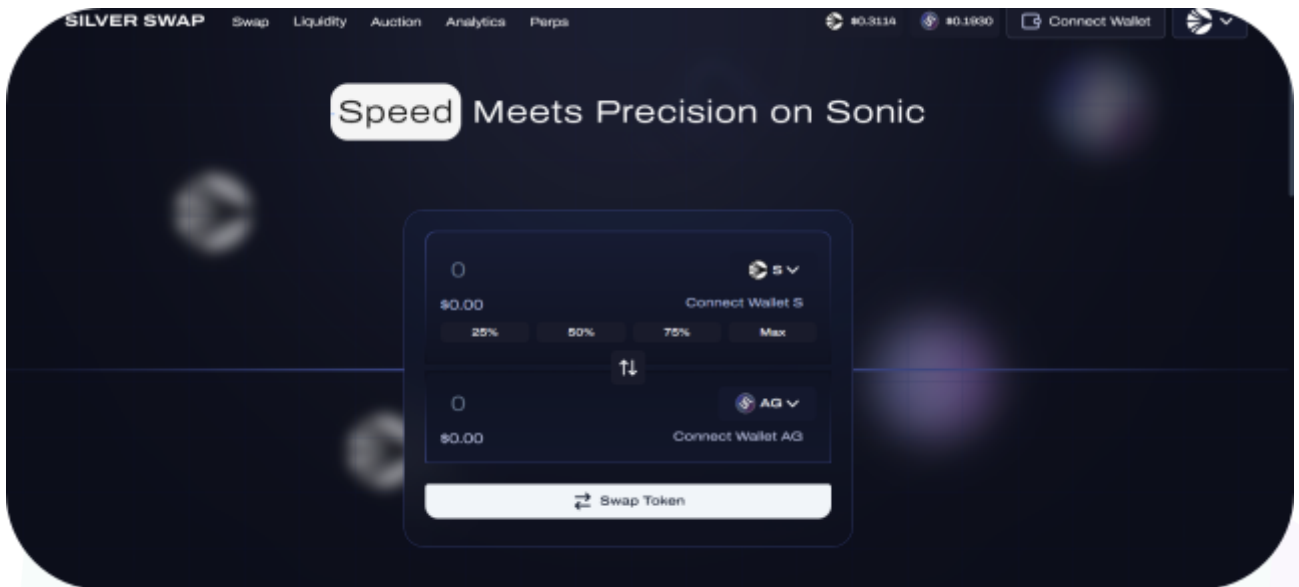
Website: <https://silverswap.io/?network=sonic>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Silverswap project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Silverswap Smart Contracts
Platform	Ethereum / Solidity
Audit Date	September, 2025
Contract 1	SilverDCA.sol
Audited GitHub Commit Hash	d3f53a4d8c9c5397af94cee871bc91be67b65bcb
Fix Review GitHub Commit Hash	43a7bc1829fdf41a92f500ec243c15488a1d6b68

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

3 High severity vulnerabilities

4 Medium severity vulnerabilities

1 Low severity vulnerability

1 Gas Optimisation

2 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
SilverDCA.sol - Users can create DCA orders on the SilverSwap platform - Orders execute at user-defined intervals with specified amounts - Comprehensive order lifecycle management (create, edit, stop, restart) - Seamless integration with Gelato - Comprehensive code documentation with NatSpec comments	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Silverswap project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Silverswap project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] SilverDCA#createOrder - Immediate Order Execution in createOrder Function

Description

Orders are executed immediately upon creation without performing the same safety checks as the `executeOrder()` function, potentially leading to failed transactions or unexpected behavior.

Vulnerability Details

In `createOrder()`, the function immediately calls `_executeOrder()` without validating user balances or allowances:

```
_executeOrder(ordersCount - 1, 0, dcaArgs); //L185
```

Impact

Users can create orders without sufficient token balance/allowance; failed order creation transactions waste gas.

Recommendation

Add necessary validation checks to `createOrder()` before executing.

Status

Resolved

[H-02] SilverDCA#_executeOrder - Arbitrary External Call Execution

Description

SilverDCA contract allows execution of arbitrary calldata on the swap router without proper validation, potentially enabling malicious actors to execute unintended functions.

Vulnerability Details

In the `_executeOrder()` function, user-controlled `dcaArgs` calldata is passed directly to the swap router:

```
(bool success, bytes memory returnData) = address(swapRouter).call(dcaArgs); //L108
```

Impact

Drainage of approved tokens and other attacks that could be carried out (or enabled with updates) using the swap router are possible.

Recommendation

Implement a function selector or other input validations where appropriate.

Status

Resolved

[H-03] SilverDCA#_executeOrder - Malicious Tokens Can Evade Fees

Description

The contract accepts any ERC20 token without restrictions and trusts the return value of `transfer()` calls for fee collection, allowing malicious tokens to evade fees entirely.

Vulnerability Details

```
require(tokenIn.transfer(tresory, fees), 'Failed to transfer fees'); //L105
```

With no token whitelist and no balance verification, a malicious token could:

- Always return true from `transfer()`
- Never actually transfer tokens to `tresory`. Pass the `require()` check and continue execution.

The malicious token can do this with an if-else block, which will execute the transfer normally on all addresses except the `tresory` address.

Impact

Loss of protocol fee revenue and unfair advantage to malicious users.

Recommendation

Add balance validation, use `safeTransfer`, and optionally add token whitelisting functionality if you want:

```
1. uint256 balanceBefore = IERC20(tresory).balanceOf(address(tokenIn));
2. SafeERC20.safeTransfer(tokenIn, tresory, fees);
3. uint256 balanceAfter = IERC20(tresory).balanceOf(address(tokenIn));
4. require(balanceAfter >= balanceBefore + fees, "Fee transfer verification failed");
```

Status

Resolved

Medium

[M-01] SilverDCA#executeOrder - Missing Slippage Protection in Fee Swaps

Description

Gelato fee swaps lack slippage protection, potentially resulting in unfavorable exchange rates and excessive fee costs.

Vulnerability Details

In `executeOrder()`, when swapping `tokenIn` to native tokens for Gelato fees, there's no minimum output validation:

```
// L147-148  
  
(bool success, bytes memory returnData) = address(swapRouter).call(gelatoFeesArgs);  
  
require(success, "Gelato fees swap failed");
```

Impact

This allows unpredictable execution costs for users and potential MEV attacks that result in the manipulation of fee swap pricing.

Recommendation

Add minimum output validation for fee swaps.

Status

Resolved

[M-02] SilverDCA#executeOrder, SilverDCA#_executeOrder - Missing Deadline Protection for Swaps

Description

Swap operations lack deadline parameters, allowing transactions to execute with potentially stale prices after delays in the mempool.

Vulnerability Details

Neither main swaps nor fee swaps don't include deadline protection:

```
(bool success, bytes memory returnData) = address(swapRouter).call(dcaArgs); // L108  
(bool success, bytes memory returnData) = address(swapRouter).call(gelatoFeesArgs);  
// L147
```

Impact

Swaps could execute with outdated prices after mempool delays.

Recommendation

Add deadline parameters to swap functions.

Status

Acknowledged

[M-03] SilverDCA#_executeOrder - Unsafe Fee Transfer Method

Description

The contract uses the unsafe `transfer()` method for fee collection instead of `SafeERC20.safeTransfer()`, which could fail silently with non-standard tokens.

Vulnerability Details

Unlike the transfer made to the user in `_executeOrder()`, `safeTransfer()` is not used in the fee transfer made to the `treasury` address.

Impact

Fee collection could fail silently with non-standard ERC20 tokens.

Recommendation

Use `safeTransfer()` methods to enhance the safety and consistency of the protocol.

Status

Resolved

[M-04] SilverDCA - No Reentrancy Protection

Description

The contract lacks reentrancy protection despite making multiple external calls, creating potential attack vectors through malicious tokens or contracts.

Vulnerability Details

Multiple external calls without any reentrancy guards:

```
require(tokenIn.transfer(treasury, fees), 'Failed to transfer fees'); // L105  
  
(bool success, bytes memory returnData) = address(swapRouter).call(dcaArgs); // L108  
  
SafeERC20.safeTransfer(tokenOut, user, amountOut); // L115
```

Impact

Malicious tokens could reenter during `transfer()` calls.

Recommendation

Add OpenZeppelin's `ReentrancyGuard` modifier in appropriate places.

Status

Resolved

Low

[L-01] SilverDCA#getOrdersByIndex - Unbounded Array Access

Description

The `getOrdersByIndex()` function doesn't validate array bounds, potentially causing reverts when accessing invalid indices.

Recommendation

Add bounds validation to prevent a poor user experience with unclear error messages.

Status

Resolved

Gas

[G-01] SilverDCA - Excessive Storage Reads

Description

The contract performs 52 separate `ordersById[id]` storage reads throughout execution, which is highly gas-inefficient. Multiple functions repeatedly access the same storage slot.

Recommendation

Load order into memory once, where possible.

Status

Resolved

QA

[Q-01] SilverDCA#onlyValidEntries - Missing Validation for amountOutMin

Description

The amountOutMin parameter can be set to 0, providing no slippage protection for users.

```
// In onlyValidEntries modifier no validation for amountOutMin  
modifier onlyValidEntries(uint256 period, uint256 amountIn, uint256 amountOutMin) {  
    require(period >= 5 minutes, 'Period must be >= 5 min');  
    require(amountIn >= 100, 'AmountIn must be > 99');  
    -;  
}
```

Recommendation

If omitting slippage protection is not intended, add validation.

Status

Acknowledged

[Q-02] SilverDCA - Typo in Variable Name**Description**

Variable `tresory` should be `treasury` for proper spelling. This might create an unprofessional code appearance.

Recommendation

Rename the variable to `treasury` throughout the contract.

Status

Resolved

Centralisation

The Silverswap project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Silverswap project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.